POC Assignment

POC1 :- ConfigMap: Volume Mount + Env Var Reference

1 Create ConfigMap app-config with keys: APP_ENV=production, APP_NAME=inventory-service.
2 Create pod config-inspector (toolbox image), that:
3 mounts app-config as a volume at /etc/app/config also injects APP_ENV as an environment variable via configMapKeyRef
3 Verify:

- kubectl exec config-inspector -- cat /etc/app/config/APP_ENV
- kubectl exec config-inspector -- printenv APP_ENV

Expected result: /etc/app/config/APP_ENV contains production. /etc/app/config/APP_NAME also exists and contains inventory-service (mounted as a whole ConfigMap, both keys appear as files). printenv APP_ENV returns production. Only APP_ENV exists as an env var — APP_NAME does not, since it was never referenced with configMapKeyRef.

```
avej_lanjekar@AvejLanjekar:~ ls
config-inspector-pod.yml  config-map.yml

avej_lanjekar@AvejLanjekar:~ cat config-map.yml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: ns-1
data:
  APP_ENV: production
  APP_NAME: inventory-service

avej_lanjekar@AvejLanjekar:~ cat config-inspector-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: config-inspector
  namespace: ns-1
spec:
  containers:
    - name: config-inspector
      image: avejlanjekar45/toolbox:1.0
```

```
      env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV
      volumeMounts:
        - name: app-config-volume
          mountPath: /etc/app/config
  volumes:
    - name: app-config-volume
      configMap:
        name: app-config

avej_lanjekar@AvejLanjekar:~ kubectl exec config-inspector -- cat
/etc/app/config/APP_ENV
production

avej_lanjekar@AvejLanjekar:~ kubectl exec config-inspector -- printenv APP_ENV
production
```

POC 2:- ConfigMap: Mounting a Single Key (subPath)

1 Reuse ConfigMap app-config from POC 1.
2 Create pod config-subpath (toolbox image) that mounts only the APP_NAME key from app-config to the path /etc/app/name.txt, using subPath.
3 Verify: kubectl exec config-subpath -- cat /etc/app/name.txt and kubectl exec config-subpath -- ls /etc/app.

Expected result: /etc/app/name.txt contains inventory-service. ls /etc/app shows only name.txt — APP_ENV is not present anywhere in the pod, confirming subPath mounts a single key instead of the whole ConfigMap.

```
avej_lanjekar@AvejLanjekar:~ kubectl get cm
NAME               DATA   AGE
app-config         2      18m
kube-root-ca.crt   1      18m

avej_lanjekar@AvejLanjekar:~ cat config-subpath-pod.yml
apiVersion: v1
kind: Pod
metadata:
```

```
  name: config-subpath
  namespace: ns-1
spec:
  containers:
    - name: config-subpath
      image: avejlanjekar45/toolbox:1.0
      volumeMounts:
        - name: config-subpath-volume
          subPath: APP_NAME
          mountPath: /etc/app/name.txt
  volumes:
    - name: config-subpath-volume
      configMap:
        name: app-config

avej_lanjekar@AvejLanjekar:~ kubectl exec config-subpath -- cat
/etc/app/name.txt
inventory-service

avej_lanjekar@AvejLanjekar:~ kubectl exec config-subpath -- ls /etc/app
name.txt
```

POC 3: Secret: Volume Mount + Env Var Reference

1 Create Secret db-credentials with keys username and password.
2 Create pod secret-inspector (toolbox image), that:
3 mounts db-credentials as a volume at /etc/app/secrets
4 injects username as env var DB_USERNAME via secretKeyRef
5 Verify:

- kubectl exec secret-inspector -- cat /etc/app/secrets/username
- kubectl exec secret-inspector -- cat /etc/app/secrets/password
- kubectl exec secret-inspector -- printenv DB_USERNAME
- kubectl exec secret-inspector -- ls -l /etc/app/secrets

Expected result: Both username and password appear as files under /etc/app/secrets with their decoded (plaintext) values. printenv DB_USERNAME returns the username value. ls -l shows the Secret volume mounted with restrictive file permissions (0644 by default, root-owned) and backed by tmpfs rather than disk.

```
avej_lanjekar@AvejLanjekar:~ ls
secret-inspector-pod.yml  secrets.yml
```

```
avej_lanjekar@AvejLanjekar:~ cat secret-inspector-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: secret-inspector
  namespace: ns-1
spec:
  containers:
    - name: secret-inspector
      image: avejlanjekar45/toolbox:1.0
      env:
        - name: DB_USERNAME
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: username
      volumeMounts:
        - name: db-credential-volume
          mountPath: /etc/app/secrets
  volumes:
    - name: db-credential-volume
      secret:
        secretName: db-credentials

avej_lanjekar@AvejLanjekar:~ cat secrets.yml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: ns-1
data:
  username: QXZlago=
  password: bGFuamVrYXIK

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-inspector -- cat
/etc/app/secrets/username
Avej

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-inspector -- cat
/etc/app/secrets/password
lanjekar

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-inspector -- printenv
DB_USERNAME
Avej
```

```
avej_lanjekar@AvejLanjekar:~ kubectl exec secret-inspector -- ls -l
/etc/app/secrets
total 0
lrwxrwxrwx    1 root     root            15 Sep  1 06:01 password ->
..data/password
```

POC 4 — Secret: Mounting a Single Key with Custom Permissions

1 Reuse Secret db-credentials.
2 Create pod secret-subpath (toolbox image) that mounts only the password key to /etc/app/secret-password, using subPath, with defaultMode: 0400.
3 Verify: kubectl exec secret-subpath -- cat /etc/app/secret-password and kubectl exec secret-subpath -- ls -l /etc/app/secret-password.

Expected result: File contains the password value only; username is not present in the pod at all. File permission mode shows 0400 (read-only for owner), confirming defaultMode was applied.

```
avej_lanjekar@AvejLanjekar:~ kubectl get secrets
NAME             TYPE     DATA   AGE
db-credentials   Opaque   2      17m

avej_lanjekar@AvejLanjekar:~ ls
secret-subpath-pod.yml

avej_lanjekar@AvejLanjekar:~ cat secret-subpath-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: secret-subpath
  namespace: ns-1
spec:
  containers:
    - name: secret-subpath
      image: avejlanjekar45/toolbox:1.0
      volumeMounts:
        - name: secret-subpath-volume
          mountPath: /etc/app/secret-password
          subPath: password
  volumes:
```

```
    - name: secret-subpath-volume
      secret:
        secretName: db-credentials
        defaultMode: 0400

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-subpath -- cat
/etc/app/secret-password
lanjekar

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-subpath -- cat
/etc/app/secret-password
lanjekar

avej_lanjekar@AvejLanjekar:~ kubectl exec secret-subpath -- ls -l
/etc/app/secret-password
-r--------    1 root     root             9 Sep  1 06:12 /etc/app/secret-
password
```

POC 5 — ServiceAccount: Assignment + Auto-Mounted Volume

1 Create ServiceAccount app-sa.
2 Create pod sa-test (toolbox image) with serviceAccountName: app-sa.
3 Verify:

- kubectl get pod sa-test -o jsonpath='{.spec.serviceAccountName}'
- kubectl exec sa-test -- ls /var/run/secrets/kubernetes.io/serviceaccount
- kubectl exec sa-test -- cat /var/run/secrets/kubernetes.io/serviceaccount/namespace

Expected result: spec.serviceAccountName returns app-sa. Without any explicit volume defined in the pod spec, /var/run/secrets/kubernetes.io/serviceaccount contains three auto-projected files: token, ca.crt, and namespace. The namespace file contains the pod's actual namespace — confirming Kubernetes injects this volume automatically for every ServiceAccount.

```
avej_lanjekar@AvejLanjekar:~ cat service-account.yml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa
  namespace: ns-1

avej_lanjekar@AvejLanjekar:~ cat sa-test-pod.yml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: sa-test
  namespace: ns-1
spec:
  serviceAccountName: app-sa
  containers:
    - name: sa-test
      image: avejlanjekar45/toolbox:1.0

avej_lanjekar@AvejLanjekar:~ kubectl get sa
NAME      AGE
app-sa    2s

avej_lanjekar@AvejLanjekar:~ kubectl get pod sa-test -o
jsonpath='{.spec.serviceAccountName}'
app-sa

avej_lanjekar@AvejLanjekar:~ kubectl exec sa-test -- ls
/var/run/secrets/kubernetes.io/serviceaccount
ca.crt
namespace
token

avej_lanjekar@AvejLanjekar:~ kubectl exec sa-test -- cat
/var/run/secrets/kubernetes.io/serviceaccount/namespace
ns-1
```

POC 6 — ServiceAccount: Disabling Auto-Mount

1 Create ServiceAccount app-sa-no-mount with automountServiceAccountToken: false.
2 Create pod sa-no-mount-test (toolbox image) using that ServiceAccount.
3 Verify: kubectl exec sa-no-mount-test -- ls /var/run/secrets/kubernetes.io/serviceaccount.

Expected result: The command fails with "No such file or directory" — no token volume is mounted, confirming automountServiceAccountToken: false on the ServiceAccount (or on the pod spec) suppresses the default auto-mount behavior.

```
avej_lanjekar@AvejLanjekar:~ ls
sa-no-mount-test-pod.yml  service-account.yml

avej_lanjekar@AvejLanjekar:~ cat service-account.yml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-sa-no-mount
  namespace: ns-1
automountServiceAccountToken: false

avej_lanjekar@AvejLanjekar:~ cat sa-no-mount-test-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: sa-no-mount-test
  namespace: ns-1
spec:
  serviceAccountName: app-sa-no-mount
  containers:
    - name: sa-no-mount-test
      image: avejlanjekar45/toolbox:1.0

avej_lanjekar@AvejLanjekar:~ kubectl get sa
NAME              AGE
app-sa            18m
app-sa-no-mount   4m29s
default           76m

avej_lanjekar@AvejLanjekar:~ kubectl exec sa-no-mount-test -- ls
/var/run/secrets/kubernetes.io/serviceaccount
ls: /var/run/secrets/kubernetes.io/serviceaccount: No such file or directory
command terminated with exit code 1
```

POC 7 — PV + PVC: Persistence Across Pod Recreation

1 Create a 1Gi PersistentVolume (hostPath is fine for a local cluster).
2 Create a PVC that binds to it.
3 Create pod pvc-test (toolbox image), mounting the PVC at /data.
4 kubectl exec pvc-test -- sh -c 'echo hello-from-poc7 > /data/hello.txt'
5 Delete pod pvc-test. Recreate it with the same PVC.
6 kubectl exec pvc-test -- cat /data/hello.txt

Expected result: /data/hello.txt still contains hello-from-poc7 after the pod is recreated. kubectl get pvc shows the claim remains Bound throughout, confirming PV/PVC lifecycle is independent of the pod lifecycle.

```
avej_lanjekar@AvejLanjekar:~ cat pvc.yml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: shared-data
  namespace: ns-1
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: nfs-csi
  resources:
    requests:
      storage: 1Gi

avej_lanjekar@AvejLanjekar:~ kubectl get pvc
NAME          STATUS   VOLUME                                     CAPACITY
ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
shared-data   Bound    pvc-623863e6-6dcc-404b-921a-729f207d1b86   1Gi
RWX            nfs-csi        <unset>                 95s

avej_lanjekar@AvejLanjekar:~ cat pvc-test-pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: pvc-test
  namespace: ns-1
spec:
  containers:
    - name: pvc-test
      image: avejlanjekar45/toolbox:1.0
      volumeMounts:
        - name: pvc-storage
          mountPath: /data
  volumes:
    - name: pvc-storage
      persistentVolumeClaim:
        claimName: shared-data

avej_lanjekar@AvejLanjekar:~ kubectl exec pvc-test -- sh -c 'echo hello-from-
poc7 > /data/hello.txt'


avej_lanjekar@AvejLanjekar:~ kubectl delete pod pvc-test
pod "pvc-test" deleted from ns-1 namespace

avej_lanjekar@AvejLanjekar:~ kubectl apply -f pvc-test-pod.yml
```

```
pod/pvc-test created

avej_lanjekar@AvejLanjekar:~ kubectl exec pvc-test -- cat /data/hello.txt
hello-from-poc7
```

POC 8:- Two Pods Sharing One PVC

1 Reuse (or create another) PVC.
2 Create pod-a and pod-b (both toolbox image), mounting the same PVC at /data.
3 pod-a: kubectl exec pod-a -- sh -c 'echo shared-data > /data/test.txt'
4 pod-b: kubectl exec pod-b -- cat /data/test.txt

Expected result: If both pods land on the same node and the PV is ReadWriteOnce, pod-b successfully reads shared-data — RWO restricts the volume to a single node, not a single pod, so co-located pods can share it. If the scheduler places pod-b on a different node, the mount fails or the pod stays stuck in ContainerCreating, since an RWO volume can't attach to two nodes at once. A ReadWriteMany PV (e.g. NFS-backed) would succeed regardless of node placement.

```
avej_lanjekar@AvejLanjekar:~ cat pod-a.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-a
  namespace: ns-1
spec:
  containers:
    - name: pod-a
      image: avejlanjekar45/toolbox:1.0
      volumeMounts:
        - name: pv-storage
          mountPath: /data
  volumes:
    - name: pv-storage
      persistentVolumeClaim:
        claimName: shared-data

avej_lanjekar@AvejLanjekar:~ cat pod-b.yml
apiVersion: v1
kind: Pod
metadata:
  name: pod-b
```

```
  namespace: ns-1
spec:
  containers:
    - name: pod-b
      image: avejlanjekar45/toolbox:1.0
      volumeMounts:
        - name: pv-storage
          mountPath: /data
  volumes:
    - name: pv-storage
      persistentVolumeClaim:
        claimName: shared-data

avej_lanjekar@AvejLanjekar:~ kubectl exec pod-a -- sh -c 'echo shared-data >
/data/test.txt'

avej_lanjekar@AvejLanjekar:~ kubectl exec pod-b -- cat /data/test.txt
shared-data
```

Poc 9:- Pod-to-Pod Communication via Service (order-service → inventory-service)

1 Create pod inventory-service using your pushed inventory-service image (from Image 2 above), that mounts ConfigMap app-config (from POC 1) as a volume at /etc/app/config, container port 5678.
2 Create a ClusterIP Service named inventory-service targeting port 5678 on that pod.
3 Create pod order-service using your pushed order-service image (from Image 3 above).
4 From order-service: kubectl exec order-service -- curl -s http://inventory-service:5678
5 Also try kubectl exec order-service -- curl -s http://:5678 — note it also works right now.
6 Delete and recreate the inventory-service pod (keep the Service and ConfigMap mount as-is). Get its new Pod IP.
7 Curl again using the old Pod IP (fails) vs the Service name (still works).

Expected result: Both curls in step 4/5 return a text response including config file APP_ENV: production. After recreating the pod in step 6, the old Pod IP curl fails (connection refused/timeout), while curl http://inventory-service:5678 in step 7 still succeeds and shows the same output — confirming the Service routes to whichever pod currently matches its selector, regardless of Pod IP changes.

avej_lanjekar@AvejLanjekar:~ ls
inventory-service-pod.yml inventory-service.yml order-service-pod.yml

avej_lanjekar@AvejLanjekar:~ cat inventory-service-pod.yml
apiVersion: v1
kind: Pod
metadata:
name: inventory-service
namespace: ns-1
labels:
app: inventory-service
spec:
containers:
- name: inventory-service
image: avejlanjekar45/inventory-service:1.0
ports:
- containerPort: 5678
volumeMounts:
- name: inventory-pv
mountPath: /etc/app/config
volumes:
- name: inventory-pv
configMap:
name: app-config

avej_lanjekar@AvejLanjekar:~ cat inventory-service.yml
apiVersion: v1
kind: Service
metadata:
name: inventory-service
namespace: ns-1
spec:
selector:
app: inventory-service
ports:
- port: 5678
targetPort: 5678
protocol: TCP

avej_lanjekar@AvejLanjekar:~ cat order-service-pod.yml
apiVersion: v1
kind: Pod
metadata:
name: order-service

namespace: ns-1
spec:
containers:
- name: order-service
image: avejlanjekar45/order-service:1.0

avej_lanjekar@AvejLanjekar:poc9$ kubectl exec -n ns-1 order-service -- curl -s http://inventory-service:5678
INVENTORY SERVICE
==========
APP_ENV env var:
DB_USERNAME env var:
config file APP_ENV: production
config file APP_NAME: inventory-serviceavej_lanjekar@AvejLanjekar:poc9$

avej_lanjekar@AvejLanjekar:~ kubectl exec order-service -- curl -s http://10.244.1.16:5678
INVENTORY SERVICE
==========
APP_ENV env var:
DB_USERNAME env var:
config file APP_ENV: production
config file APP_NAME: inventory-service

avej_lanjekar@AvejLanjekar:~ kubectl delete -f inventory-service-pod.yml
pod "inventory-service" deleted from ns-1 namespace

avej_lanjekar@AvejLanjekar:~kubectl apply -f inventory-service-pod.yml
pod/inventory-service created

avej_lanjekar@AvejLanjekar:~ kubectl exec order-service -- curl -s http://10.244.1.16:5678
command terminated with exit code 28

avej_lanjekar@AvejLanjekar:~ kubectl exec -n ns-1 order-service -- curl -s http://inventory-service:5678
INVENTORY SERVICE
==========
APP_ENV env var:
DB_USERNAME env var:
config file APP_ENV: production
config file APP_NAME: inventory-serviceavej_lanjekar@AvejLanjekar:poc9$